DIPLOMA EN CIENCIA DE DATOS
E INTELIGENCIA ARTIFICIAL APLICADA

RedNova

Ingeniería de Datos, pronósticos y operación con agentes

Versiones, reproducibilidad y Docker

Módulo 6 · Semana 4 · Clase teórica del lunes

Prof. Mag. Ing. Mat. Diego Fernández

La pregunta que organiza el libro

¿Cómo integrar ventas, inventarios, costos y pronósticos para proponer una reposición que otra persona pueda comprender, revisar y reproducir?

Clase de 90 minutos y lectura de ampliación.

Caso ficticio; importes en unidades monetarias ficticias.

Cómo utilizar este libro

RedNova es un nuevo caso integrador de la Semana 4. Recupera habilidades estudiadas previamente —definir contratos, revisar datos, comprender código generado por agentes y exigir evidencia— y las aplica a un problema de mayor escala y complejidad. No supone que los trabajos de semanas anteriores ya contenían este caso ni reescribe su historia.

El lunes se estudia el diseño de un sistema: qué problema resuelve, qué datos necesita, cómo se combinan, qué cálculos son deterministas, dónde interviene un pronóstico y cómo se controla la participación de agentes. Docker y el versionado aparecen como respuestas a necesidades de ese sistema. El objetivo es comprender y justificar decisiones, sin memorizar comandos ni implementar una plataforma industrial durante la clase.

La presentación conduce los 90 minutos; este libro desarrolla las explicaciones con mayor profundidad. Las pausas son preguntas de discusión y los microejemplos están resueltos. No se incluye una agenda de laboratorio, una consigna de entrega ni una modificación de la evaluación. La práctica del miércoles se preparará posteriormente sobre una porción acotada del mismo caso.

Qué existe en este material y qué se está proponiendo

RedNova, sus volúmenes y sus datos son ficticios. Los nombres de archivos describen **entradas previstas del sistema**; no indican que esos archivos hayan sido suministrados o cargados. Las tablas pequeñas son extractos sintéticos para razonar. Los pronósticos y métricas ilustran cálculos, sin afirmar entrenamiento, evaluación de producción ni ejecución integral del pipeline. Los comandos son plantillas conceptuales, no registros de ejecución.

Dos niveles de lectura. El núcleo del lunes conecta problema, granularidad, uniones, cálculo, reposición, pronóstico, versiones, Docker y agentes. La ampliación profundiza contratos, temporalidad, incidentes, manifiestos y límites. Puede consultarse después sin convertir cada detalle en una nueva exigencia presencial.

Qué debe poder explicar el estudiante. De dónde sale una cifra; qué registros y reglas la determinan; por qué una unión puede duplicarla; qué información estaba disponible al pronosticar; cómo se obtiene una propuesta de reposición; qué cambió entre versiones y qué evidencia sería necesaria para aceptar el resultado.

Guion conceptual de 90 minutos

Minutos	Pregunta que organiza la conversación	Resultado de comprensión
0–10	¿Qué decisión necesita RedNova?	Distinguir informar, pronosticar y proponer.
10–25	¿Qué representa cada fila y cómo se relaciona?	Reconocer grano, claves, contratos y uniones.
25–40	¿Cómo se calculan margen y reposición?	Seguir los ejemplos numéricos y sus límites.
40–53	¿Cómo sabemos si un pronóstico ayuda?	Comprender evaluación temporal y ciclo MLOps.
53–65	¿Qué debemos conservar para repetir?	Identificar familias de versiones y corrida.
65–78	¿Qué ejecuta Docker y a qué puede acceder?	Explicar imagen, montajes y permisos.
78–86	¿Qué delegamos y qué revisamos?	Dirigir agentes y diagnosticar fallos.
86–90	¿Qué justificaría aceptar el resultado?	Reconstruir la cadena de evidencia.

Índice general

Cómo utilizar este libro **i**

Guion conceptual de 90 minutos . . . II

1. El problema de RedNova **1**

1.1. La organización y su necesidad . . . 1

1.2. Tres objetivos que deben convivir . . . 1

1.3. Escala de diseño, no evidencia de archivos cargados . . . 2

1.4. La conexión con Ingeniería de Datos, MLOps y agentes . . . 2

2. Fuentes y granularidad **3**

2.1. La granularidad precede a la herramienta . . . 3

2.2. Hechos, dimensiones y estados . . . 4

2.3. Ventas netas y significado del descuento . . . 4

2.4. Inventario, reservas y tránsito . . . 4

3. Contratos y calidad **6**

3.1. Un contrato expresa significado y condiciones de uso . . . 6

3.2. Tres niveles de calidad que conviene separar . . . 6

3.3. Duplicado exacto y conflicto de identidad . . . 6

3.4. Un resultado parcial debe presentarse como parcial . . . 7

3.5. Rechazar datos no significa perderlos . . . 7

4. Tiempo y disponibilidad **8**

4.1. Evento, ingesta, corte y decisión . . . 8

4.2. Vigencias de costos y de políticas . . . 8

4.3. Reprocesar no es reescribir la historia . . . 8

4.4. Faltante, cero y censura por falta de stock . . . 9

5. Uniones e integración **10**

5.1. Una unión relaciona filas; no comprende el negocio . . . 10

5.2. Cardinalidad y controles antes y después . . . 10

5.3. La unión temporal con costos . . . 10

5.4. El problema muchos a muchos entre flujos . . . 11

5.5. Integrar inventario después de definir el grano . . . 11

6. Margen explicado **12**

6.1. El extracto sintético de S03 . . . 12

6.2. Cálculo de una línea y del conjunto . . . 12

6.3. Una discrepancia que revela una unión incorrecta . . . 13

6.4. El costo más reciente puede alterar indebidamente el pasado . . . 13

6.5. Precisión, redondeo y significado . . . 13

7. Reposición razonada **14**
7.1. Las magnitudes que intervienen . . . 14
7.2. La política de cobertura . . . 14
7.3. Cálculo resuelto con v1 . . . 15
7.4. Qué demuestra cambiar únicamente la política . . . 15
7.5. La aritmética no comprueba el calendario de abastecimiento . . . 15
7.6. Seguridad didáctica y límite del modelo . . . 16

8. Pronosticar con evidencia **17**
8.1. Qué se pretende predecir . . . 17
8.2. M0: un baseline que puede explicarse . . . 17
8.3. M1: un candidato supervisado . . . 17
8.4. Variables disponibles y fuga de información . . . 18
8.5. El registro de un pronóstico . . . 18

9. Evaluación temporal **19**
9.1. Separar aprendizaje, selección y evaluación final . . . 19
9.2. Ventanas completas y fronteras entre particiones . . . 19
9.3. Por qué un reparto aleatorio puede engañar . . . 19
9.4. MAE y WAPE en cuatro observaciones sintéticas . . . 20
9.5. Qué permiten concluir estas métricas . . . 20

10. Arquitectura del proceso **21**
10.1. Un pipeline es una secuencia con contratos entre etapas . . . 21
10.2. Tres capas con funciones distintas . . . 21
10.3. Productos mínimos y propósito . . . 22
10.4. Determinismo e idempotencia . . . 22
10.5. Una corrida como unidad de reconstrucción . . . 23

11. Versiones y reproducibilidad **24**
11.1. De cinco piezas sencillas a familias explícitas . . . 24
11.2. Qué significa obtener el mismo resultado . . . 24
11.3. Git como memoria del trabajo técnico . . . 24
11.4. Huellas, identificadores y procedencia . . . 25
11.5. Cambiar una pieza permite formular una hipótesis . . . 25
11.6. Semillas y límites de reproducibilidad . . . 25

12. Docker en RedNova **26**
12.1. Qué problema resuelve el entorno . . . 26
12.2. El agente permanece fuera del contenedor . . . 26
12.3. Una imagen no es una computadora completa . . . 26
12.4. Dockerfile conceptual explicado . . . 27
12.5. Contexto pequeño y exclusiones . . . 27

12.6. Etiqueta, ID y digest 27

13. Montajes y permisos **28**

13.1. Qué es un montaje 28
13.2. Leer las opciones de ejecución por su propósito 28
13.3. Permisos del anfitrión y usuario del contenedor 29
13.4. Red de construcción y red de ejecución 29
13.5. Recursos y salidas persistentes 29

14. Agentes con responsabilidades **30**

14.1. Un agente actúa dentro de herramientas y permisos 30
14.2. Roles que ordenan el trabajo 30
14.3. Un prompt de diseño para el lunes 30
14.4. Un prompt de revisión que busca refutar errores 31
14.5. Cuatro estados que deben distinguirse 31
14.6. Las fuentes aportan contenido, no autoridad nueva 32

15. Operación y cambios **33**

15.1. Operar significa observar y responder 33
15.2. La calidad predictiva se conoce con demora 33
15.3. Cambio de distribución y deterioro de desempeño 33
15.4. Promover una versión de forma controlada 33
15.5. Tres incidentes y tres investigaciones diferentes 34

16. Cierre conceptual **35**

16.1. Recorrer la cadena desde la decisión hasta la fuente 35
16.2. Preguntas que cierran el lunes 35
16.3. Continuidad hacia la práctica 35

A. Mapa de decisiones y glosario **36**

A.1. Una lectura compacta de los controles 36
A.2. Vocabulario para releer el diseño 36

B. Ficha conceptual de una corrida **38**

Referencias y recursos **39**

Capítulo 1

Un sistema para informar y proponer decisiones

1.1 La organización y su necesidad

RedNova es una red ficticia de distribución y comercio con 12 sucursales, 120 productos distribuidos en seis categorías y un centro abastecedor. Cada noche necesita consolidar ventas y margen, conocer disponibilidad y proponer reposición. La finalidad es reducir faltantes sin acumular inventario innecesario, conservando una explicación de cómo se obtuvo cada recomendación.

El problema es más amplio que producir un gráfico. Las ventas llegan como líneas de transacciones; los inventarios describen estados al cierre; las recepciones registran movimientos ya ocurridos; los pedidos abiertos representan compromisos todavía no completados; los costos cambian en el tiempo. Cada fuente responde una pregunta diferente. Juntarlas sin respetar esas diferencias puede producir una tabla prolija y una conclusión equivocada.

La dirección no necesita únicamente saber cuánto se vendió. Debe distinguir si un producto vendió poco porque hubo poca demanda o porque no estuvo disponible. Tampoco puede interpretar toda mercadería pedida como disponible hoy. El sistema debe mostrar límites cuando las fuentes no permiten resolver esas preguntas, evitando convertir ausencia de información en una cifra aparentemente precisa.

La solicitud profesional

«Preparar el cierre de ventas y margen, informar disponibilidad por sucursal y producto, y proponer unidades a reponer para los próximos siete días. Identificar faltantes de información, conservar versiones y mostrar qué datos y reglas sostienen cada propuesta. La compra requiere revisión y aprobación del responsable».

El resultado esperado es una *propuesta*, no una orden automática. El responsable debe revisar presupuesto, capacidad física, cantidades mínimas del proveedor, promociones y cambios conocidos del negocio. Una cantidad calculada con una fórmula no contiene por sí sola esas decisiones.

1.2 Tres objetivos que deben convivir

El primer objetivo es de servicio: contar con productos cuando se necesitan. El segundo es económico: evitar capital inmovilizado y exceso de existencias. El tercero es de confiabilidad: saber cuándo los datos permiten sostener una conclusión. Estos objetivos pueden entrar en tensión. Aumentar todas las coberturas puede reducir ciertos faltantes, pero también elevar inventario; mantener coberturas bajas puede liberar capital y aumentar riesgo de quiebre.

No asignaremos valores inventados al costo de un faltante o al costo financiero del inventario. La política didáctica utilizará días de cobertura y un pronóstico de siete días. Esa simplificación permite entender el proceso, pero no transforma la fórmula en una solución óptima para la empresa. La discusión profesional consiste en reconocer qué se modeló y qué decisiones permanecen fuera.

1.3 Escala de diseño, no evidencia de archivos cargados

Para dimensionar el escenario consideramos los 365 días de 2025, 12 sucursales y 120 productos. La grilla sucursal–producto–día tendría

$$365 \times 12 \times 120 = 525600$$

combinaciones previstas. Esta cifra representa posiciones posibles de una tabla analítica diaria, no 525600 transacciones realizadas. En una combinación puede haber varias ventas, ninguna venta o información incompleta.

Suponemos aproximadamente 180000 líneas de venta y 12000 líneas de recepción anuales como escala ilustrativa. No son conteos obtenidos de archivos existentes. El volumen es suficiente para que copiar, comparar y unir datos manualmente sea una estrategia frágil, pero no exige por sí solo una arquitectura distribuida. Un diseño claro y herramientas ordinarias pueden ser adecuados antes de introducir más infraestructura.

1.4 La conexión con Ingeniería de Datos, MLOps y agentes

La Ingeniería de Datos se ocupa del recorrido que convierte fuentes heterogéneas en productos utilizables: ingesta, estructura, validación, integración, transformación y procedencia. MLOps agrega prácticas para desarrollar, evaluar, desplegar y operar modelos de aprendizaje automático a lo largo de su ciclo de vida [1]. En RedNova ambas disciplinas se encuentran en el pronóstico y en las condiciones que permiten utilizarlo.

La operación de agentes incorpora otra pregunta: ¿qué instrucciones, herramientas y acciones contribuyeron al producto? El agente puede preparar código, explicar un contrato o investigar discrepancias. No debe sustituir el cálculo verificable por intuiciones de lenguaje ni decidir una política comercial sin autoridad. El sistema tendrá partes deterministas, un componente predictivo cuando exista un modelo aprobado y una revisión humana que acepte el uso.

Idea central

RedNova permite estudiar un sistema completo sin confundir sus funciones: los datos describen hechos y estados; el pronóstico estima una magnitud futura; la política transforma esa estimación en una propuesta; la persona responsable decide qué acción corresponde.

Capítulo 2

Comprender qué representa cada fila

2.1 La granularidad precede a la herramienta

La granularidad, o grano, es el nivel al que se define una fila. Una línea de venta, un cierre diario de inventario y una predicción de siete días no representan la misma unidad de observación. Antes de unir archivos debemos completar una frase: «cada fila representa exactamente...». Si no podemos hacerlo, tampoco sabemos qué clave identifica una fila ni qué significa contarla.

Una clave es un conjunto de campos que identifica una observación dentro de la fuente. En ventas se utiliza la pareja `venta_id,linea_id`: una venta puede contener varios productos o líneas, de modo que el identificador de venta por sí solo no es único. En inventario, la combinación es `fecha,sucursal_id,producto_id`. La clave expresa el grano y sirve para comprobar duplicados y conflictos.

Las fuentes siguientes son un diseño de entradas. El origen indicado describe un sistema funcional hipotético; no afirma una conexión existente con un software comercial.

Fuente y tipo	Origen previsto y grano	Clave y periodicidad	Control central
`ventas_lineas.csv` Hechos de venta	Sistema de ventas; una línea de transacción	Venta y línea; ingesta por lote, consolidación diaria	Clave única; cantidad, precio y descuento válidos.
`inventario_diario.csv` Estado	Sistema de inventario; cierre de una sucursal y producto en una fecha	Fecha, sucursal, producto; diario	Stock no negativo; reserva no mayor que stock.
`recepciones.csv` Hechos de movimiento	Recepción física; una línea recibida	Recepción y línea; por lote	Cantidad y referencia a línea de pedido coherentes.
`pedidos_abiertos.csv` Estado de pedidos	Gestión de abastecimiento; una línea pedida	Pedido y línea; corte diario	Recibido no mayor que pedido; estado y arribo.
`productos.csv` Maestro	Catálogo; un producto	Producto; cuando cambia	Identificador único, categoría y actividad.
`sucursales.csv` Maestro	Organización; una sucursal	Sucursal; cuando cambia	Identificador único y región.
`costos_vigentes.csv` Regla temporal de gestión	Área responsable de costos; un período por producto	Producto y comienzo de vigencia; cuando cambia	Períodos sin solapamientos; costo no negativo.
`politica_reposicion.json` Configuración	Responsable comercial; una versión de política	Versión y vigencia; cuando cambia	Regla aprobada y única para decisión.

Fuente y tipo	Origen previsto y grano	Clave y periodicidad	Control central
`pronosticos.csv` Producto de modelo	Proceso predictivo; una predicción por emisión, sucursal, producto y horizonte	Emisión, sucursal, producto, horizonte y versión de modelo; periódica	Modelo y corte identificados; sin información futura.

Cada línea de recepción conserva su propia clave `recepcion_id,linea_id`. Para conciliarla con un pedido necesita además `pedido_id,pedido_linea_id`: el identificador de línea de recepción no debe confundirse con la línea del pedido. Varias recepciones parciales pueden corresponder a una misma línea de pedido, mediante una relación muchos a uno.

2.2 Hechos, dimensiones y estados

Un hecho registra una ocurrencia o magnitud de actividad. Las ventas y recepciones son flujos: ocurren durante un período. Una dimensión describe entidades y permite organizarlas; productos y sucursales son maestros que aportan categoría, región o estado activo. Su propósito no es añadir importes sino dar significado a las claves.

El inventario diario es un estado observado en un momento. Sumar el stock de lunes, martes y miércoles no produce el stock del miércoles: cuenta repetidamente existencias que pueden haber permanecido en la tienda. Es correcto sumar stock entre productos o sucursales compatibles para una misma fecha, según el propósito; sumar entre fechas exige definir otra medida, como un promedio de existencias.

Una distinción que evita dobles conteos

Si el cierre de inventario ya incorpora la recepción del día, sumar nuevamente esa recepción al stock de cierre contaría dos veces el mismo movimiento. La recepción sirve para explicar y conciliar el cambio del stock, no para aumentar automáticamente un estado ya consolidado.

2.3 Ventas netas y significado del descuento

Cada línea de ventas incluye sucursal, producto, fecha y hora del evento, cantidad, precio unitario, descuento monetario de toda la línea y fecha y hora de ingesta. El contrato define

$$\text{venta neta} = \text{cantidad} \times \text{precio unitario} - \text{descuento de línea}.$$

El descuento no es un porcentaje ni un importe por unidad. Para veinte unidades de precio 50 y descuento de línea 100, la venta neta es $20 \times 50 - 100 = 900$, no 800 ni cero. La claridad semántica evita que dos implementaciones correctas en sintaxis calculen negocios distintos.

2.4 Inventario, reservas y tránsito

El disponible se define como stock de cierre menos stock reservado. La reserva es una parte del stock que ya tiene un compromiso y no debe ofrecerse libremente. El tránsito elegible es otra magnitud: cantidades pendientes de pedidos confirmados, no cancelados, cuya llegada prevista se encuentra dentro del horizonte considerado. No representa una recepción física ya completada.

Una fuente faltante no se sustituye automáticamente por cero. Si falta la fila de inventario, no sabemos si el producto quedó sin stock o si la extracción falló. Si no conocemos la fecha de arribo de un pedido, no podemos incluirlo silenciosamente en la disponibilidad del horizonte. El sistema debe preservar esa incertidumbre.

Para discutir: ¿Por qué contar filas de ventas y contar filas de inventario responde preguntas diferentes aunque ambos archivos contengan producto y sucursal?

Capítulo 3

Definir qué datos permiten sostener un resultado

3.1 Un contrato expresa significado y condiciones de uso

Un contrato de datos define estructura, tipos, claves, dominios, reglas de validez y tratamiento de incumplimientos. No es únicamente una lista de columnas. También establece qué significa una ausencia, qué decisiones pueden automatizarse y qué problemas exigen revisión. El agente necesita este contrato antes de generar transformaciones; de lo contrario deberá completar vacíos con supuestos.

Para ventas, la cantidad debe ser un entero positivo. Las devoluciones quedan fuera del caso y no se deducen de valores negativos. El descuento de línea debe encontrarse entre cero y el importe bruto de la línea. Los importes y cantidades deben poder interpretarse de forma finita y coherente con sus unidades. Un registro que viola una regla no se arregla cambiando el signo o reemplazando un campo sin autorización.

Para inventario, stock de cierre y reservado son enteros no negativos y la reserva no puede superar el stock. Para pedidos, la cantidad recibida acumulada no debe superar la cantidad pedida. La operación $\max(\text{pedida} - \text{recibida}, 0)$ se aplica a registros previamente válidos; no se utiliza para esconder un conflicto donde recibido es mayor que pedido.

3.2 Tres niveles de calidad que conviene separar

La validez de fila comprueba una observación: formato, cantidad, rango o fecha. La integridad referencial comprueba relaciones entre fuentes: el producto de una venta debe existir en el maestro correspondiente. La calidad del lote examina propiedades del conjunto: completitud, esquema compatible, claves únicas y cobertura temporal.

Una fila puede ser válida por sí sola y no poder integrarse. Por ejemplo, una venta con cantidad y precio correctos puede referirse a un producto desconocido. Asimismo, todas las filas pueden ser válidas individualmente y existir dos filas maestras para la misma clave, lo que vuelve ambigua la unión. La calidad del sistema no puede reducirse al porcentaje de celdas completas.

3.3 Duplicado exacto y conflicto de identidad

Un duplicado exacto repite la misma clave y el mismo contenido. Se conserva la primera aparición identificada y se registra la repetida, manteniendo procedencia. Un conflicto repite la clave con contenido distinto; no debe resolverse eligiendo una fila arbitrariamente. Debe bloquearse la partición afectada y solicitar revisión del origen o de la regla autorizada.

La partición es una unidad explícita de procesamiento: por ejemplo, un grupo de sucursal, producto y día. Su definición permite limitar el impacto de un problema sin esconderlo. Si el conflicto no puede localizarse de forma confiable, el alcance de la detención debe ampliarse. El agente no debe decidir qué parte es prescindible para terminar un informe.

Problema	Respuesta prevista	Consecuencia del resultado
Duplicado exacto	Conservar primero y registrar repetido.	Resultado trazable bajo regla explícita.
ID con contenido conflictivo	Bloquear partición afectada y revisar.	No publicar esa partición como completa.
Esquema global incompatible	Detener la etapa afectada.	No continuar con renombrados intuitivos.
Maestro con claves duplicadas	Detener unión.	Relación muchos a uno no garantizada.
Producto desconocido o costo ausente	Bloquear grupo afectado y reportar.	Producto incompleto o no evaluable.
Períodos de costo solapados	Bloquear aplicación de costo al grupo.	No escoger último costo por conveniencia.

3.4 Un resultado parcial debe presentarse como parcial

Puede ser útil conservar productos de grupos válidos mientras otros permanecen bloqueados. Pero el total general no puede presentarse como completo si omite grupos no procesados. El informe debe mostrar alcance, grupos incluidos, grupos bloqueados y motivo. Un total parcial puede servir para investigar; no reemplaza el cierre completo solicitado.

Por ejemplo, si A100 no tiene un costo aplicable único, sus ventas pueden estar observadas, pero su margen no es evaluable. No corresponde omitir A100 y llamar «margen total de la sucursal» al agregado de B200 y C300. Tampoco corresponde asignar costo cero: eso fabricaría rentabilidad. La forma de presentar el producto debe seguir su nivel real de completitud.

3.5 Rechazar datos no significa perderlos

Los registros originales se conservan en una capa inmutable, con identificador de lote y huella cuando esta se obtiene realmente. Los rechazos son productos separados que incluyen clave, origen, regla y motivo. Así podemos corregir una interpretación o recibir una rectificación sin reconstruir a mano qué se descartó.

La revisión humana se concentra en las decisiones que el contrato no resuelve. El agente puede identificar el problema y ofrecer alternativas, pero no debe normalizar silenciosamente una nueva categoría, deducir una devolución o elegir un costo entre varios. Un proceso que se detiene correctamente puede ser más confiable que otro que siempre produce números.

Idea central

La calidad consiste en determinar qué afirmaciones permiten los datos. Una regla de validación debe conducir a una decisión visible sobre el alcance del resultado, no únicamente a un contador de errores.

Capítulo 4

Cuatro fechas que no responden la misma pregunta

4.1 Evento, ingesta, corte y decisión

La fecha del evento indica cuándo ocurrió una venta o recepción. La fecha de ingesta indica cuándo esa información estuvo disponible para el sistema. La fecha de corte delimita qué información puede utilizar una corrida. La fecha de decisión determina qué política corresponde aplicar a una propuesta. Aunque algunas coincidan, representan funciones distintas.

Esta separación es necesaria tanto para reconstruir informes históricos como para evaluar modelos. Una venta ocurrida el 30 de junio puede llegar al sistema el 1 de julio. Desde el punto de vista del negocio pertenece al 30; desde el punto de vista de una predicción emitida antes de su llegada, todavía no estaba disponible. Usarla retroactivamente como si se hubiera conocido genera una evaluación engañosa.

Un cierre con información tardía

La corrida con corte al final del 30/06/2025 en UTC utiliza los datos disponibles hasta ese corte. Una venta del 30 ingresada el 01/07 genera una nueva corrida o corrección identificada cuando se incorpora. La corrida histórica no se reescribe de manera silenciosa: se conserva qué sabía el sistema en ese momento.

El tiempo no es solo un formato de columna. Deben definirse zona horaria, inclusividad de límites y reglas de llegada tardía. En este libro los cortes mencionados utilizan UTC para evitar ambigüedad. En una implementación se documentaría además cómo convertir horas locales de los sistemas de origen.

4.2 Vigencias de costos y de políticas

El costo estándar de gestión se selecciona por fecha de venta. Cada intervalo utiliza comienzo inclusivo y fin exclusivo: [desde, hasta). El fin abierto indica que no se definió aún un cierre para ese período. Para cada línea de venta debe existir exactamente un costo aplicable.

La política de reposición se selecciona por fecha de decisión. En el caso, v1 rige hasta el 30/06/2025 inclusive y v2 desde el 01/07/2025. Esto no implica que una venta histórica de junio deba valorarse con el costo o la política más nueva. Cada regla debe relacionarse con la fecha que el contrato le asigna.

4.3 Reprocesar no es reescribir la historia

Un reprocesamiento puede tener distintos propósitos: repetir exactamente una corrida, incorporar datos tardíos, corregir un error de código o estudiar una política alternativa. Cada propósito produce una relación distinta con el resultado anterior. Debe quedar registrado qué cambió y por qué, sin utilizar la misma etiqueta para ocultar una modificación.

Cuando incorporamos una venta tardía, el nuevo resultado puede ser una corrección del cierre. Cuando mantenemos los datos y cambiamos una política, obtenemos un escenario comparativo. Cuando corregimos código, debemos explicar qué salidas anteriores estaban afectadas. Estas distinciones permiten comunicar cambios de forma responsable.

4.4 Faltante, cero y censura por falta de stock

Una observación cero de ventas es información: el sistema registró ausencia de ventas bajo condiciones que deben conocerse. Un día faltante es ausencia de información. Completar todos los faltantes con cero puede enseñar al modelo que no hubo actividad donde en realidad no hubo datos.

Además, ventas observadas y demanda no son idénticas. Si no había stock, vender cero no demuestra demanda cero: pudo existir demanda no satisfecha. A este problema se lo denomina posible censura por disponibilidad. En RedNova los días con quiebre se etiquetan para advertir que la venta observada puede no representar toda la demanda.

La política analítica didáctica evita imputaciones silenciosas. El baseline solo se aplica con siete días completos y sin quiebre señalado. Si esas condiciones no se cumplen, se informa que el método no tiene entrada válida, en lugar de fabricar una estimación mediante ceros. Un modelo más avanzado necesitaría un tratamiento explícito y evaluado de esta limitación.

Para discutir: ¿Por qué una variable con fecha de evento anterior al corte todavía podría ser inválida para una predicción emitida en ese corte?

Capítulo 5

Combinar fuentes sin fabricar ventas

5.1 Una unión relaciona filas; no comprende el negocio

Una unión combina filas según condiciones entre claves. Su resultado depende de cuántas coincidencias encuentra para cada observación. La herramienta no sabe si el aumento de filas representa detalle legítimo o una duplicación accidental. Por eso necesitamos definir la relación esperada antes de ejecutarla.

Ventas con productos y ventas con sucursales requieren relaciones muchos a uno. Puede haber muchas ventas del mismo producto, pero debe haber una única fila del producto en el maestro aplicable. Si el maestro contiene dos registros con la misma clave, una venta puede duplicarse al unir. La regla de unicidad es parte de la corrección del cálculo, no una preferencia de orden.

5.2 Cardinalidad y controles antes y después

La cardinalidad describe cuántas observaciones de una fuente pueden corresponder a una de la otra. Antes de unir comprobamos unicidad de claves maestras y claves de hechos. Después contamos filas, coincidencias ausentes y coincidencias múltiples. En una unión de enriquecimiento muchos a uno, esperamos conservar el número de líneas de venta válidas y sus importes.

Si la unión encuentra un producto desconocido, no conviene perder esa fila silenciosamente con una operación que conserve solo coincidencias. Debe detectarse y bloquearse el grupo afectado según el contrato. Una reducción de filas no es necesariamente una limpieza; puede ser pérdida no explicada de actividad.

Un control que acompaña la unión

Antes de agregar categoría, existen tres líneas válidas con ventas netas 2900. Después de unir al maestro de productos deberían seguir existiendo tres líneas y las mismas ventas. Si aparecen cuatro filas y ventas 3800, se investiga la multiplicación antes de seguir calculando margen o reposición.

5.3 La unión temporal con costos

Para cada venta se busca el producto correspondiente y el intervalo cuyo comienzo es menor o igual a la fecha del evento y cuyo fin es posterior a ella, o permanece abierto. La condición conceptual es

$$\text{producto coincide}, \qquad \text{desde} \le \text{fecha de venta} < \text{hasta}.$$

No basta con ordenar costos por fecha y elegir el último. Tampoco basta con unir solo por producto: el producto puede tener legítimamente varios períodos históricos. El control exige exactamente una coincidencia por línea de venta.

Si no hay coincidencia, falta un costo para ese evento. Si hay dos, los períodos se solapan o la fuente

tiene un conflicto. Ambos casos bloquean el margen del grupo afectado y deben reportarse. Aplicar una media de costos o conservar una coincidencia arbitraria cambia la regla de negocio.

5.4 El problema muchos a muchos entre flujos

Supongamos que un producto tiene tres líneas de venta y dos líneas de recepción el mismo día. Si se unen ambos archivos solo por sucursal, producto y fecha, pueden aparecer seis combinaciones: cada venta coincide con cada recepción. Sumar después las ventas las repite dos veces; sumar recepciones las repite tres.

La solución conceptual consiste en agregar primero cada flujo al grano requerido. Las ventas diarias se resumen por fecha, sucursal y producto; las recepciones también. Recién entonces se relacionan productos con claves únicas a ese nivel. La agregación debe conservar sus propias medidas y no mezclar importes de naturaleza distinta.

Regla para leer cualquier unión

Declarar el grano de ambas entradas, la clave de unión y la cardinalidad esperada. Comprobar unicidad donde corresponda, medir coincidencias y contrastar conteos e importes antes y después. Si los controles fallan, la existencia de una tabla de salida no demuestra integración correcta.

5.5 Integrar inventario después de definir el grano

La tabla diaria combina una venta agregada con un estado de inventario de la misma fecha, sucursal y producto. Un día con inventario y sin ventas registradas puede ser una observación válida de ventas cero únicamente si el proceso garantiza que la fuente de ventas está completa. Si el lote no llegó, la ausencia no debe interpretarse de esa manera.

Los pedidos y recepciones requieren además considerar estado y tiempo. Una recepción efectiva reduce el pendiente del pedido; el stock de cierre ya consolidado la incorpora según el proceso de origen. El sistema debe evitar sumar simultáneamente cantidades recibidas, stock consolidado y cantidades pedidas completas como si todas fueran disponibilidad adicional.

Idea central

La integración correcta conserva significado, no solo columnas. Las claves, la cardinalidad y el tiempo determinan qué sumas son legítimas y qué resultados deben bloquearse.

Capítulo 6

Un cierre pequeño que permite revisar el cálculo

6.1 El extracto sintético de S03

El siguiente extracto describe tres líneas válidas de la sucursal S03 con fecha de evento 30/06/2025. Su función es permitir un contraste manual antes de pensar en el panel anual. No representa un lote ejecutado ni resume las 180000 líneas supuestas del escenario.

Producto	Categoría	Cantidad	Precio	Descuento	Costo unitario
A100	Oficina	20	50	100	30
B200	Hogar	5	200	0	150
C300	Tecnologia	2	500	0	400

El costo de la tabla es el costo estándar de gestión aplicable a la fecha de cada venta. No se trata de un método de valuación contable del inventario. Esta delimitación evita atribuir al ejemplo reglas como costo promedio o primera entrada–primera salida que no forman parte del contrato.

6.2 Cálculo de una línea y del conjunto

Para una línea con cantidad q, precio p, descuento monetario d y costo aplicable c, definimos

$$V = qp - d, \qquad C = qc, \qquad M = V - C.$$

En A100, el importe bruto es $20 \times 50 = 1000$; al restar el descuento de línea 100 se obtienen ventas netas 900. El costo es $20 \times 30 = 600$ y el margen 300. El descuento reduce ventas netas; no modifica la cantidad ni autoriza a descontar nuevamente el costo.

B200 produce ventas 1000, costo 750 y margen 250. C300 produce ventas 1000, costo 800 y margen 200. La suma de las tres líneas es:

Producto	Unidades	Ventas netas	Costos	Margen
A100	20	900	600	300
B200	5	1000	750	250
C300	2	1000	800	200
Total	27	2900	2150	750

El porcentaje global de margen es

$$100\frac{750}{2900} = 25{,}8620689655\ldots\ \%,$$

que se presenta como 25,86 %. Se divide la suma de márgenes por la suma de ventas netas. No se promedian sin ponderación los porcentajes de cada producto, porque cada línea participa con un importe de ventas diferente.

6.3 Una discrepancia que revela una unión incorrecta

Imaginemos que A100 encuentra dos costos por períodos solapados. Una unión ingenua genera dos filas para esa venta. Al sumar ventas se obtiene $2900 + 900 = 3800$, aunque no ocurrió ninguna venta nueva. El aumento de 900 es un efecto de integración.

No calculamos un margen erróneo total en este escenario porque no se especificaron los dos costos coincidentes. Atribuirle un valor sería inventar datos. Lo que sí podemos demostrar es la duplicación de ventas y la violación del requisito de una única coincidencia temporal. Debe bloquearse el costo de A100 y evitar presentar el total de margen como completo.

Este ejemplo enseña por qué conviene conservar controles previos al modelo. Ningún algoritmo de pronóstico ni contenedor corrige una unión que multiplicó transacciones. La calidad del resultado depende primero de la representación del problema y de sus transformaciones.

6.4 El costo más reciente puede alterar indebidamente el pasado

Para A100, supongamos costo 30 vigente en $[2025\text{-}01\text{-}01, 2025\text{-}07\text{-}01)$ y costo 32 desde 01/07/2025, con fin abierto. La venta del 30 de junio utiliza 30 incluso si se vuelve a procesar en julio. Seleccionar «el costo más nuevo» daría costo $20 \times 32 = 640$ y margen $900 - 640 = 260$: una caída artificial de 40 respecto del margen correcto 300.

El cambio no describe una pérdida comercial ocurrida entre ejecuciones. Describe una implementación que seleccionó otra referencia temporal. Si el negocio aprobara corregir un costo histórico, se necesitaría una versión rectificada y una explicación del reprocesamiento; no sería una consecuencia automática de que exista un costo posterior.

6.5 Precisión, redondeo y significado

Los importes se calculan con aritmética decimal y una política de precisión explícita. En Python, `Decimal` permite representar decimales a partir de sus cadenas y controlar operaciones y redondeo [11]. Eso no corrige fórmulas equivocadas ni reemplaza las pruebas; reduce una fuente de diferencias cuando se utiliza de manera adecuada.

El redondeo de presentación ocurre al mostrar el importe o porcentaje, no en cada paso intermedio sin justificación. En reposición se aplicará techo para obtener unidades enteras según una fórmula definida. Son decisiones diferentes: los importes monetarios y las unidades de producto no comparten necesariamente la misma regla de redondeo.

Para discutir: Si dos informes muestran el mismo porcentaje redondeado, ¿qué campos y reglas tendrían que revisarse antes de declarar que el cálculo coincide?

Capítulo 7

Transformar un pronóstico en una propuesta revisable

7.1 Las magnitudes que intervienen

Una propuesta de reposición relaciona demanda futura estimada, inventario disponible, cantidades en tránsito y una política de cobertura. Ninguna de esas magnitudes reemplaza a las demás. El pronóstico no es una venta ya ocurrida; el stock no es un flujo diario; el tránsito no es mercadería necesariamente disponible hoy.

Para S03/A100, el ejemplo hipotético de cierre del 30 de junio utiliza una demanda residual prevista hipotética de 140 unidades para los próximos siete días. Stock de cierre 80 y reservado 10 producen disponible $80 - 10 = 70$. Las diez unidades reservadas corresponden a compromisos previos que están **fuera** de esas 140 unidades. Antes de aplicar la fórmula se concilia el pronóstico total con los compromisos ya reservados para obtener demanda residual. No se resta de nuevo una reserva que ya quedó fuera de esa magnitud. Si se desconoce el solapamiento entre reservas y pronóstico, el estado es `REVISAR`.

Suponemos además 40 unidades pendientes de pedidos confirmados, no cancelados y con llegada futura dentro del horizonte. Esa cantidad es tránsito elegible. Antes de usarla se necesita comprobar el pendiente de cada línea y la fecha prevista. Un pedido atrasado con fecha pasada no se vuelve elegible solo porque su fecha sea menor que el final del horizonte: requiere confirmar una nueva llegada útil.

7.2 La política de cobertura

La política v1 utiliza horizonte de siete días y cobertura de seguridad de dos días. La política v2 aumenta esta última a tres días. La demanda diaria media utilizada por la regla se define como

$$\bar{d} = \frac{\widehat{D}_7}{7},$$

donde $\widehat{D}_7$ representa aquí la demanda residual prevista de los próximos siete días, después de conciliar compromisos reservados. No es una demanda observada de cada día: es una distribución uniforme de la magnitud agregada para esta heurística.

Con s días de seguridad, las cantidades se calculan como

$$\begin{aligned} SS &= \left\lceil \bar{d}\, s \right\rceil, \\ O &= \left\lceil \widehat{D}_7 + SS \right\rceil, \\ A &= \text{stock de cierre} - \text{reservado}, \\ P &= A + T, \\ Q &= \max(0, O - A - T), \end{aligned}$$

donde SS es la cobertura de seguridad expresada en unidades, O el objetivo, A el disponible, T el

tránsito elegible, P la posición considerada y Q la propuesta provisional de reposición.

El techo $\lceil x \rceil$ redondea hacia el siguiente entero cuando es necesario, pues no se proponen fracciones de unidad bajo el contrato. El máximo con cero evita cantidades negativas de reposición: si la posición supera el objetivo, la regla propone cero. Eso no equivale a desconocer el resultado.

7.3 Cálculo resuelto con v1

La demanda diaria media es $140/7 = 20$. Con dos días de cobertura, $SS = \lceil 20 \times 2 \rceil = 40$. El objetivo es $\lceil 140 + 40 \rceil = 180$. El disponible 70 más el tránsito 40 da posición 110. Por tanto,

$$Q = \text{máx}(0, 180 - 70 - 40) = 70.$$

Las 70 unidades son una cantidad provisional derivada de supuestos explícitos. No constituyen una compra aprobada. La propuesta debe acompañarse de la fecha de decisión, versión de política, referencia del pronóstico y datos de inventario y pedidos utilizados.

Magnitud	Política v1	Escenario controlado v2
Demanda residual de siete días	140	140
Demanda diaria media	20	20
Días de cobertura de seguridad	2	3
Seguridad en unidades	40	60
Objetivo	180	200
Disponible	70	70
Tránsito elegible	40	40
Propuesta provisional	70	90

7.4 Qué demuestra cambiar únicamente la política

Si mantenemos todas las magnitudes y cambiamos a tres días de seguridad, $SS = 60$, objetivo 200 y propuesta $200 - 70 - 40 = 90$. La diferencia de veinte unidades se explica por el cambio de política. Es una comparación contrafactual controlada: no afirmamos que el inventario o el pronóstico reales del 1 de julio sean iguales a los del día anterior.

En operación, v1 está vigente hasta el 30 de junio inclusive y v2 desde el 1 de julio. La fecha de decisión selecciona la política, pero también debe utilizarse información válida para esa decisión. El experimento conceptual conserva todo lo demás para aislar el efecto de la regla; una corrida real posterior necesita observar sus entradas.

7.5 La aritmética no comprueba el calendario de abastecimiento

Con 70 unidades disponibles y consumo supuesto uniforme de 20 por día, la cobertura inmediata sería $70/20 = 3{,}5$ días. Aunque existan 40 unidades que llegarán dentro de la semana, podría ocurrir un faltante antes de su arribo. Asimismo, las unidades propuestas podrían llegar demasiado tarde si el nuevo pedido tiene un plazo largo.

Por eso la propuesta agregada debe contrastarse con las fechas concretas de llegada y el plazo del nuevo pedido. Si esas fechas son ambiguas o no permiten juzgar la continuidad de abastecimiento, el

estado es `REVISAR`, aunque podamos mostrar el cálculo provisional 70. Una cifra aritméticamente definida no implica una recomendación operativamente aceptable.

7.6 Seguridad didáctica y límite del modelo

La cobertura fija en días es una heurística sencilla. No estima una probabilidad de servicio ni demuestra un stock óptimo. No considera de manera completa variabilidad de demanda, incertidumbre del plazo, correlaciones, sustitución entre productos o costos económicos del faltante y del inventario.

Su utilidad pedagógica es hacer visibles las dependencias. Si el pronóstico cambia, cambia el objetivo; si aumenta la reserva, disminuye disponible; si un arribo deja de estar confirmado, disminuye tránsito elegible. El responsable debe revisar también mínimos de compra, capacidad, presupuesto, promociones y cambios de surtido antes de aprobar una acción.

Cero, no evaluable y revisión

Propuesta cero: las entradas son válidas y la regla no requiere reposición.
No evaluable: falta una magnitud necesaria o no se puede aplicar la regla.
REVISAR: existe información incompleta, conflictiva o una condición operativa sin resolver. Puede acompañar un cálculo provisional, que no debe presentarse como propuesta aceptada.

Capítulo 8

De una referencia simple a un modelo candidato

8.1 Qué se pretende predecir

El objetivo predictivo es la demanda de siete días por sucursal y producto. La disponibilidad de datos exige una precisión: se observan ventas, mientras la demanda no satisfecha puede quedar sin registrar. El diseño debe reconocer esa diferencia antes de llamar «demanda real» a toda suma de ventas.

Los días con quiebre se etiquetan como posiblemente censurados. Las observaciones faltantes también se identifican. El modelo no debe aprender que la falta de datos equivale a poca demanda ni que la falta de stock demuestra ausencia de interés. El tratamiento de estos casos debe formar parte de la configuración, las pruebas y la interpretación del pronóstico.

8.2 M0: un baseline que puede explicarse

El baseline M0 utiliza la media de ventas diarias de los siete días anteriores observados y la multiplica por siete:

$$\widehat{D}_7^{(M0)} = 7 \left(\frac{1}{7} \sum_{j=1}^{7} y_{t-j+1} \right).$$

Se aplica únicamente cuando esos siete días están completos y no tienen quiebre señalado. Su sencillez permite comprender cómo produce una estimación y constituye una referencia que un modelo más complejo debería justificar que mejora.

Si los siete valores son 8, 10, 12, 9, 11, 7 y 13 unidades, suman 70; su media es 10 y el pronóstico de siete días es 70. Este extracto es ilustrativo y muestra la operación; no afirma que el pronóstico del caso haya sido generado por un pipeline real.

M0 y M1 estiman ventas o demanda total bajo sus condiciones. Esa salida no debe utilizarse como demanda residual sin conciliar los compromisos ya reservados. Las 70 unidades de este ejemplo aritmético del baseline y las 140 residuales del ejemplo de reposición cumplen funciones diferentes y son ejemplos independientes; no representan una conciliación realizada.

El baseline supone que la semana reciente es informativa sobre la próxima. Puede fallar ante promociones, estacionalidad, cambios de surtido o rupturas. Esas limitaciones no lo vuelven inútil: establecen las condiciones bajo las cuales compararemos alternativas. Si falta uno de los días, no se sustituye por cero para hacer funcionar la fórmula.

8.3 M1: un candidato supervisado

Un modelo supervisado aprende una relación entre variables disponibles en una fecha de emisión y un resultado futuro conocido en el conjunto de entrenamiento. Un candidato basado en árboles puede combinar rezagos de ventas, calendario, promociones conocidas al emitir y señales de disponibilidad. Su salida sería una estimación de unidades para los siete días siguientes.

Un rezago es un valor pasado relativo a la fecha de emisión. Una variable de calendario puede indicar día de semana o período del año. Una promoción solo puede utilizarse si estaba planificada y disponible para el sistema en esa fecha. La información de stock ayuda a interpretar ventas observadas, pero no elimina por sí misma la censura de demanda.

El entrenamiento consiste en ajustar el modelo con ejemplos históricos. La inferencia consiste en utilizar una versión ya ajustada para producir predicciones nuevas. En el lunes se estudia esa distinción; no se afirma que se entrenó un candidato ni se exige hacerlo durante la clase.

8.4 Variables disponibles y fuga de información

Existe fuga de información cuando el modelo o su evaluación utilizan datos que no habrían estado disponibles en el momento de la predicción, o cuando el proceso incorpora de forma indebida información del resultado. Puede producir métricas excelentes y desempeño decepcionante al operar.

Un ejemplo claro sería incorporar ventas de los próximos siete días entre las variables de entrada. Otros errores son menos visibles: utilizar una revisión posterior de inventario, una promoción registrada después de la emisión o una transformación ajustada con todo el período de prueba. También pueden introducirse costos posteriores o información ingresada después del corte, aunque su fecha de evento sea antigua.

La disponibilidad debe documentarse para todas las fuentes mediante lotes, fechas de disponibilidad y versiones de snapshot. No alcanza con tener fecha de ingesta en ventas si el maestro, inventario o pedidos fueron sustituidos por una versión posterior. Reconstruir una predicción histórica requiere reconstruir lo que el sistema podía conocer.

8.5 El registro de un pronóstico

El producto previsto `pronosticos.csv` contiene fecha de emisión, sucursal, producto, horizonte, demanda prevista, identificador y versión del modelo, versión de variables, corte de datos y fecha máxima de la información utilizada. La clave debe distinguir emisión, sucursal, producto, horizonte y versión de modelo.

Puede haber pronósticos de M0 y M1 para el mismo grupo cuando se comparan candidatos. El proceso de reposición debe seleccionar **una única emisión disponible al corte y una única versión aprobada** por sucursal, producto y horizonte. Una regla explícita puede elegir la última emisión válida disponible; debe conservarse cuál eligió y por qué. Unir ambos modelos o varias emisiones con inventario y sumar propuestas duplicaría el producto analítico. La selección de la versión aprobada es parte de la configuración de la corrida.

Idea central

Un pronóstico útil tiene más identidad que un número: necesitamos saber para qué período se emitió, qué podía conocerse, qué modelo lo produjo y qué limitaciones afectan su interpretación.

Capítulo 9

Comparar modelos sin conocer el futuro

9.1 Separar aprendizaje, selección y evaluación final

El entrenamiento ajusta el modelo; la validación ayuda a elegir configuración y comparar candidatos; la prueba final estima comportamiento sobre un período que no guio esas elecciones. Mezclar estas funciones hace difícil saber cuánto de la mejora es aprendizaje útil y cuánto es adaptación a los datos de evaluación.

En un diseño temporal ilustrativo, el entrenamiento utiliza información hasta el 31 de marzo, la validación abril y la prueba mayo; junio representa una etapa posterior de operación. Estos intervalos ordenan el tiempo, pero deben aplicarse respetando las ventanas objetivo de siete días. No basta con asignar una observación según la fecha de origen y permitir que su objetivo invada la partición siguiente.

9.2 Ventanas completas y fronteras entre particiones

Cada ejemplo tiene una fecha de emisión y un objetivo formado por los siete días posteriores. Su ventana objetivo debe quedar completa dentro del período al que pertenece. Las observaciones cercanas a la frontera que mezclen objetivos entre entrenamiento, validación y prueba se excluyen o purgan según el diseño. Por ejemplo, un origen al final de marzo cuyo objetivo alcanza abril no pertenece sin más a un entrenamiento cuyo resultado disponible termina en marzo.

Las variables también deben calcularse con el corte correspondiente a cada origen. Los parámetros de transformaciones aprendidas —por ejemplo, una normalización ajustada a datos— se estiman solo con la información permitida del entrenamiento. La separación temporal debe preservarse en toda la cadena, no únicamente al final del modelo.

La validación con ventanas temporales permite repetir el análisis en varios períodos. Herramientas como `TimeSeriesSplit` ilustran este enfoque [12], pero su parámetro `gap` cuenta muestras, no necesariamente días. En un panel con muchos productos y sucursales por fecha, ordenar filas y dejar siete muestras de separación no equivale a dejar siete días. El diseño debe agrupar y delimitar por fecha de forma explícita.

9.3 Por qué un reparto aleatorio puede engañar

Si se mezclan aleatoriamente observaciones del mismo período, entrenamiento y prueba pueden compartir información muy próxima o variables construidas con datos futuros. La comparación deja de representar la pregunta de operación: predecir lo que todavía no ocurrió con lo que se sabe hoy. El problema es especialmente importante en objetivos de varios días que se solapan.

La evaluación temporal tampoco garantiza por sí sola ausencia de fuga. Si las fuentes históricas se reconstruyen con revisiones posteriores, todavía puede haber información no disponible en su momento. Necesitamos tanto el orden correcto de períodos como versiones de datos y transformaciones coherentes con cada corte.

9.4 MAE y WAPE en cuatro observaciones sintéticas

Consideremos cuatro predicciones comparables con reales 100, 200, 50 y 150. Estos valores ilustran el cálculo de métricas; no corresponden a una evaluación anual de RedNova ni permiten una conclusión estadística sobre producción.

Real	M0	Error absoluto M0	M1	Error absoluto M1
100	90	10	98	2
200	220	20	210	10
50	40	10	55	5
150	160	10	147	3
500		50		20

El error absoluto de una predicción es $|y - \widehat{y}|$. El error absoluto medio, MAE, es

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \widehat{y}_i|.$$

Para M0, $MAE = 50/4 = 12{,}5$ unidades; para M1, $MAE = 20/4 = 5$ unidades. Esta métrica expresa el tamaño medio del error en la unidad del objetivo.

El error porcentual absoluto ponderado, WAPE, se calcula como

$$WAPE = 100\frac{\sum_i |y_i - \widehat{y}_i|}{\sum_i |y_i|}.$$

Con reales no negativos que suman 500, M0 tiene $100(50/500) = 10\,\%$ y M1 $100(20/500) = 4\,\%$ [13]. No llamamos a este último resultado «96 % de accuracy»: es otra métrica y no equivale a una probabilidad de acierto.

9.5 Qué permiten concluir estas métricas

En las cuatro observaciones, M1 tiene menor error bajo ambas medidas. Eso demuestra el cálculo comparativo del ejemplo; no demuestra superioridad estadísticamente significativa ni desempeño del panel anual. Para elegir una versión de producción necesitaríamos evidencia más amplia y representativa.

El agregado puede ocultar diferencias por producto, región, promociones o disponibilidad. Un modelo puede mejorar en productos de gran volumen y empeorar en otros importantes para servicio. WAPE pondera mediante el volumen del denominador agregado; MAE también necesita interpretación respecto de escalas y grupos. Si la suma de reales es cero, WAPE no está definido; MAE sí puede calcularse.

La selección considera además disponibilidad de datos, latencia, costo, estabilidad y riesgo de uso. Una mejora pequeña puede no justificar una gran complejidad adicional. El baseline conserva valor como referencia, detector de anomalías y posible alternativa cuando el candidato no cumple sus condiciones de operación.

Para discutir: Si M1 mejora WAPE global, ¿qué información adicional pedirías antes de usar sus predicciones para proponer reposición en todas las sucursales?

Capítulo 10

Organizar capas y productos que puedan revisarse

10.1 Un pipeline es una secuencia con contratos entre etapas

El pipeline de RedNova organiza ingesta, normalización, validación, uniones, agregación diaria, construcción de variables, pronóstico, aplicación de política y elaboración de propuestas e informe. Cada etapa recibe entradas definidas y produce salidas que la siguiente puede comprobar. Esta organización permite localizar fallos y repetir una parte sin confundirla con el proceso completo.

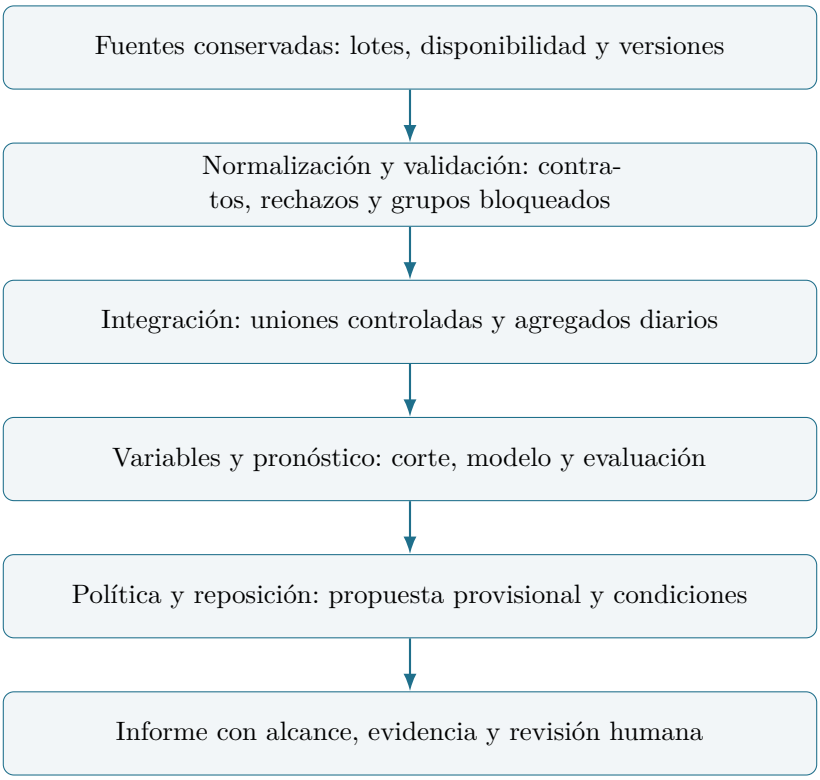
Una secuencia de instrucciones no constituye por sí sola un pipeline confiable. Necesitamos saber qué ocurre si una etapa falla, qué productos quedan incompletos y qué resultados pueden utilizarse. Si la cobertura de costos es insuficiente, el informe de margen no debe publicarse como completo aunque el proceso de inventario haya terminado correctamente.

10.2 Tres capas con funciones distintas

La capa *raw* conserva los datos tal como llegaron, con origen, lote, disponibilidad y huellas calculadas cuando corresponda. Su inmutabilidad protege la capacidad de reconstruir. No significa que los datos sean correctos; significa que el original recibido queda disponible para entender qué ocurrió.

La capa validada contiene datos normalizados conforme a contratos, registros aceptados y reportes de rechazos o grupos bloqueados. La normalización puede convertir representaciones de fechas o tipos cuando el contrato lo autoriza; no debe modificar el significado comercial. Dentro de esta etapa puede existir un área *stage* de trabajo para transformaciones intermedias, sin confundirla con una fuente aceptada.

La capa de productos contiene agregados diarios, variables, pronósticos, propuestas e informes. Cada producto conserva referencias a las entradas y a la corrida. Un informe es una vista de esos productos; no debería convertirse en la única evidencia que sobrevive.



El esquema muestra dependencias, no una obligación de construir seis servicios separados. Un programa bien organizado puede expresar estas etapas. La complejidad técnica debe responder a una necesidad, no al deseo de incorporar más componentes.

10.3 Productos mínimos y propósito

Un diagnóstico de calidad explica qué se aceptó, qué se bloqueó y por qué. Un agregado diario reúne medidas compatibles con el grano. Un registro de pronóstico identifica modelo y corte. Una propuesta de reposición explica magnitudes y política. El informe comunica esas piezas de manera comprensible, incluyendo límites.

Las salidas deben distinguir datos observados de estimados. No conviene presentar una columna de ventas históricas y otra de pronóstico sin indicar cuál pertenece al futuro. Tampoco conviene sustituir un pronóstico faltante por cero para completar una tabla. El diseño del producto protege al usuario de interpretaciones que el cálculo no sostiene.

10.4 Determinismo e idempotencia

Las transformaciones de contratos, uniones y fórmulas deben producir los mismos resultados relevantes cuando las entradas y condiciones son las mismas. Eso es una expectativa de determinismo que se comprueba. La inferencia de un modelo aprobado también debe registrarse y evaluarse bajo sus condiciones de reproducibilidad; ciertos componentes numéricos pueden depender de plataforma o configuración.

La idempotencia responde otra pregunta: ¿repetir una operación acumula efectos adicionales? Un cálculo puede devolver siempre 70 y, sin embargo, crear una nueva orden cada vez que se ejecuta. El número sería estable, pero la acción duplicaría compras. En RedNova el pipeline produce propuestas,

no envía pedidos. Si posteriormente se diseñara publicación operativa, necesitaría claves y reglas para evitar duplicaciones.

Guardar una carpeta por corrida conserva historia, pero no garantiza idempotencia de una acción externa. La distinción evita creer que versionar archivos controla automáticamente todos los efectos del sistema.

10.5 Una corrida como unidad de reconstrucción

Una corrida es una ejecución identificada del proceso con entradas, parámetros y versiones concretas. Su manifiesto relaciona el identificador de corrida, corte, fecha de decisión, fuentes, contratos, código, modelo, política, entorno, productos y estado. Los valores no observados se marcan pendientes; no se inventan hashes ni identificadores para dar apariencia de trazabilidad.

Si una corrida falla, el manifiesto puede conservar estado y diagnóstico sin productos finales aceptados. Si produce solo algunos grupos, registra alcance parcial. El estado del proceso es parte de la información que necesita el usuario del informe.

Capítulo 11

Conservar la combinación que produjo una propuesta

11.1 De cinco piezas sencillas a familias explícitas

En problemas pequeños es útil distinguir datos, código, instrucciones, política y entorno. RedNova agrega modelado, múltiples contratos y diversas configuraciones. Conviene ampliar esa organización a familias explícitas, sin forzar todos los artefactos dentro de cinco casillas.

Familia	Qué se conserva y por qué
Fuentes y contratos	Lotes, disponibilidad, snapshots, claves y reglas para interpretar el dato.
Transformaciones y variables	Código de integración, agregación y construcción de features.
Modelo y desarrollo	Artefacto aprobado, configuración de entrenamiento, datos, evaluación y límites.
Configuración y políticas	Horizonte, cobertura, vigencias, selección de modelo y parámetros de corrida.
Instrucciones y herramientas	Prompts, alcance de agentes y herramientas efectivamente utilizadas.
Entorno	Intérprete, dependencias, plataforma e imagen observada.
Manifiesto de corrida	Vínculo entre todas las familias, entradas, productos y revisión.

Una versión del informe puede depender de versiones diferentes de cada familia. No es necesario renombrar todo como v2 porque cambió una instrucción. Lo necesario es conocer la combinación y conservar el estado anterior cuando se modifica una pieza.

11.2 Qué significa obtener el mismo resultado

Para comparar ejecuciones debemos definir equivalencia. En calidad de datos interesan registros y grupos aceptados o bloqueados; en integración, claves y agregados; en margen, importes y reglas temporales; en pronóstico, modelo, corte y valores; en reposición, cantidades, política y condiciones. La redacción del informe puede variar sin alterar esas propiedades.

También pueden existir diferencias de representación que no cambien significado. Dos JSON con distinto orden de claves pueden contener los mismos campos y valores. Pero `null` no equivale a cero ni a la cadena `"null"`. Las unidades, fechas, tipos y precisión importan. Comparar únicamente capturas o porcentajes redondeados puede ocultar discrepancias.

11.3 Git como memoria del trabajo técnico

Git permite registrar estados de archivos, compararlos y conservar su historia. Un repositorio reúne ese historial; un commit identifica un estado seleccionado junto con un mensaje; un diff muestra modificaciones; una rama permite trabajar una alternativa antes de incorporarla a la versión aceptada [2, 3].

En RedNova, Git resulta apropiado para código, contratos, configuración y textos de instrucciones. Los datos de mayor volumen y los artefactos de modelo pueden conservarse en almacenamiento separado, con referencias y huellas en el manifiesto. No se trata de incorporar indiscriminadamente todo al repositorio. Tampoco se necesita publicar un repositorio para comprender o usar control de versiones.

Un commit no demuestra que un modelo sea bueno ni que una unión sea correcta. Conserva un estado para que las pruebas y la revisión puedan referirse a él. De forma similar, volver a una versión de código no deshace una compra emitida ni corrige automáticamente un informe ya comunicado.

11.4 Huellas, identificadores y procedencia

Una huella de contenido ayuda a comprobar si un archivo cambió. Su valor debe calcularse realmente y conservarse con el método y el archivo al que corresponde. No reemplaza la procedencia: una huella no explica quién produjo el dato, cuándo estuvo disponible ni qué contrato lo interpreta.

La identidad del modelo también debe ser explícita. «Modelo nuevo» no permite distinguir entrenamiento, variables o configuración. Una versión registrada relaciona artefacto, datos de desarrollo, parámetros, transformaciones, métricas y decisión de aprobación. Las métricas ilustrativas del capítulo anterior no constituyen ese registro real.

11.5 Cambiar una pieza permite formular una hipótesis

Si mantenemos datos y pronóstico y cambiamos cobertura de dos a tres días, el efecto controlado sobre la propuesta del ejemplo es 70 a 90. Si cambia también el inventario, ya no podemos atribuir toda diferencia a la política. La comparación debe identificar qué cambió realmente.

Si se corrige una unión temporal, pueden cambiar márgenes históricos aunque los datos de ventas permanezcan iguales. Si se cambia el modelo, pueden cambiar propuestas aunque política y stock sean idénticos. Si se modifica el prompt del agente, puede cambiar la explicación sin cambiar el cálculo. Esta separación ayuda a diagnosticar, comunicar y decidir qué pruebas repetir.

11.6 Semillas y límites de reproducibilidad

Una semilla controla parte de la aleatoriedad de ciertos procedimientos de entrenamiento. Ayuda a repetir condiciones, pero no garantiza por sí sola resultados idénticos en cualquier versión de biblioteca, hardware o configuración de paralelismo. Es necesario conservar el entorno y comprobar el comportamiento que importa para el uso.

La imagen de Docker tampoco congela todo el hardware o kernel. Una referencia de contenido mejora la identidad del paquete, mientras el manifiesto completa plataforma y parámetros. La reproducibilidad es una propiedad que se diseña y comprueba; no una promesa automática de una herramienta.

Idea central

Versionar permite saber qué se utilizó. Reproducir exige además reconstruir las condiciones relevantes y contrastar productos. Aceptar exige decidir que esa evidencia alcanza para el propósito concreto.

Capítulo 12

Separar el programa de sus entradas y salidas

12.1 Qué problema resuelve el entorno

Un script puede funcionar en una computadora y fallar en otra porque cambian versiones, dependencias, rutas o permisos. Docker permite preparar una imagen que contiene el programa y sus dependencias, y ejecutar instancias de esa imagen bajo condiciones explícitas [4]. Su aporte es organizar el entorno; no corrige contratos ni garantiza calidad del modelo.

El Dockerfile describe cómo se prepara una imagen. La imagen es el paquete reutilizable. El contenedor es una instancia de ejecución con entradas, parámetros y accesos concretos. Construir una imagen y ejecutar un contenedor son acciones distintas; ambas necesitan evidencia propia cuando se realizan.

En RedNova elegimos una imagen con **código y dependencias**; datos, configuración y modelo aprobado se suministran externamente. Así una nueva entrada diaria no exige reconstruir la imagen y las familias de versiones se identifican por separado. El modelo externo solo existe cuando se ha preparado y aprobado; su ausencia debe tratarse explícitamente, no reemplazarse por un supuesto artefacto.

12.2 El agente permanece fuera del contenedor

El agente ayuda a preparar y supervisar el proceso desde su entorno de trabajo. El contenedor batch ejecutaría transformaciones deterministas y, cuando exista un modelo aprobado, su inferencia. No se necesita un modelo de lenguaje ni una API dentro de la imagen para calcular las magnitudes definidas.

Un proceso batch recibe un conjunto de entradas, trabaja y termina. No requiere publicar una página web ni mantener un servicio expuesto para esta tarea. Por esa razón el diseño no publica puertos y puede ejecutar el cálculo sin red externa si todas las entradas y dependencias necesarias están disponibles.

12.3 Una imagen no es una computadora completa

Los contenedores Linux utilizan un kernel Linux disponible para el motor. En instalaciones habituales sobre macOS o Windows, ese entorno se proporciona mediante virtualización según la instalación. El contenedor no incorpora cualquier kernel ni elimina toda relación con el anfitrión.

La arquitectura puede ser `arm64` o `amd64`. Algunas imágenes ofrecen variantes; el motor selecciona una compatible o puede utilizar emulación. Por eso la etiqueta de una imagen no describe por sí sola todas las condiciones de ejecución. Plataforma, versión de Python y referencias observadas se registran cuando afectan la comparación.

12.4 Dockerfile conceptual explicado

La plantilla siguiente propone una organización de código y una interfaz. No afirma que `src.pipeline`, el archivo de dependencias o la configuración ya existan. En la implementación deberán crearse o adaptarse al programa real y comprobarse antes de ejecutar.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.lock /app/requirements.lock
RUN pip install --no-cache-dir -r requirements.lock
COPY src/ /app/src/
USER 10001:10001
ENTRYPOINT ["python", "-m", "src.pipeline"]
CMD ["--config", "/config/ejecucion.json"]
```

`FROM` selecciona una base ilustrativa oficial con Python; la etiqueta es mutable y no equivale a un contenido congelado. `WORKDIR` fija el directorio del programa. `COPY` incorpora las dependencias declaradas y el código. `RUN` instala dependencias durante la construcción. `USER` define una identidad sin privilegios administrativos para la ejecución. `ENTRYPOINT` indica el programa; `CMD` aporta argumentos predeterminados [6].

El archivo `requirements.lock` deberá contener versiones resueltas de dependencias directas y transitivas cuando se implemente. Su nombre no garantiza que esté correctamente fijado. Si se utiliza verificación con hashes, estos deben obtenerse y comprobarse realmente. No se escriben valores inventados para aparentar reproducibilidad.

La elección entre biblioteca estándar, pandas u otras herramientas se justifica según operaciones y escala. RedNova no impone una prohibición universal de dependencias. Lo importante es comprender qué aporta cada una, fijar versiones compatibles y evitar incorporar paquetes innecesarios.

12.5 Contexto pequeño y exclusiones

El contexto de construcción es el conjunto de archivos que el constructor puede utilizar [5]. Debe ser una carpeta específica del proyecto, con Dockerfile, código y dependencias previstas. No debe ser la carpeta personal ni una colección indiscriminada de documentos.

`.dockerignore` ayuda a excluir datos, resultados, archivos temporales y otros contenidos que no necesitan entrar en la imagen. La elección de entradas externas significa que los lotes de ventas y el modelo no se copian automáticamente durante la construcción. La revisión del contexto comprueba esa separación antes de construir.

12.6 Etiqueta, ID y digest

Una etiqueta como `rednova-pipeline:v1` es un nombre que puede reasignarse. Un ID observado identifica una imagen local; un digest de repositorio es una referencia de contenido cuando la herramienta la proporciona. Son conceptos relacionados, pero no intercambiables. Una imagen local puede no tener RepoDigests y esa ausencia se declara.

Fijar referencias de contenido mejora el control de cambios de la base [9]. Sin embargo, reconstruir una imagen requiere también dependencias, contexto y condiciones identificadas. Ejecutar la misma imagen y reconstruir desde un Dockerfile son dos preguntas de reproducibilidad diferentes.

Capítulo 13

Dar acceso solo a las entradas necesarias

13.1 Qué es un montaje

Un montaje vincula un archivo o directorio externo con una ruta dentro del contenedor. Permite suministrar datos sin incorporarlos a la imagen y conservar salidas después de que finaliza la instancia. Esa conexión también concede acceso: debe definirse qué carpeta se expone y con qué permisos [10].

En RedNova se propone la siguiente organización. Todas las rutas del anfitrión serán directorios específicos y autorizados del caso ficticio. Las rutas indicadas dentro del contenedor son convenciones de diseño, no evidencia de que existan carpetas preparadas.

Ruta interna	Permiso	Propósito
`/datos`	Solo lectura	Lotes de fuentes y snapshots del corte.
`/config`	Solo lectura	Contratos, política y configuración de corrida.
`/modelo`	Solo lectura	Artefacto aprobado y metadatos, cuando exista.
`/salida`	Lectura y escritura	Productos y registros de la corrida; único destino persistente de escritura.

Un montaje de solo lectura evita que ese proceso modifique el contenido a través de esa ruta; no anonimiza los datos ni los vuelve adecuados para cualquier destinatario. El programa todavía puede leerlos. La selección de datos permitidos sigue siendo necesaria.

13.2 Leer las opciones de ejecución por su propósito

La siguiente línea es **pseudocomando**: el texto entre corchetes describe grupos de opciones que deben sustituirse por montajes reales revisados. No debe copiarse como si fuera una orden completa.

```
docker run --rm --network none --read-only --cap-drop ALL --security-opt no-new-
    privileges [montajes de entrada :ro y salida :rw] rednova-pipeline:v1
```

`-rm` elimina la instancia al terminar. `-network none` evita conectividad externa del contenedor y conserva loopback [8]. `-read-only` hace de solo lectura el sistema de archivos raíz de la instancia; los montajes de escritura definidos expresamente siguen cumpliendo su función. `-cap-drop ALL` retira capacidades Linux adicionales y `no-new-privileges` impide adquirir privilegios nuevos mediante ciertos mecanismos [7].

Estas opciones no garantizan seguridad absoluta ni sustituyen la revisión de código y entradas. Expresan que una tarea de cálculo no necesita administrar el sistema, comunicarse con servicios externos ni escribir en directorios ajenos. Si una biblioteca necesita un área temporal, el diseño deberá preverla de forma acotada y comprobarla; no se amplían permisos por intuición.

13.3 Permisos del anfitrión y usuario del contenedor

La identidad `10001:10001` debe poder leer las entradas y escribir en el directorio de salida autorizado. En Linux, propietarios y permisos del anfitrión pueden afectar esa posibilidad. El agente debe inspeccionar y explicar la situación, proponiendo una configuración específica para la carpeta del caso.

No corresponde resolver un error recurriendo automáticamente a usuario administrador o permisos `777`. Primero se determina qué operación falló y qué permiso necesita. Aumentar privilegios puede ocultar una ruta equivocada o una dependencia mal diseñada en vez de corregirla.

Nunca se monta la carpeta personal completa ni el socket de Docker como atajo. El socket puede permitir controlar el motor y ampliar de forma sustancial el alcance del proceso. La tarea de RedNova necesita archivos del caso, no control general del anfitrión.

13.4 Red de construcción y red de ejecución

La construcción puede requerir red para obtener la base y las dependencias. La ejecución posterior puede realizarse sin red si dispone localmente de todo lo necesario. Desactivar red en `run` no prueba que la construcción haya ocurrido sin conectividad ni que el agente externo esté desconectado.

El agente puede tener herramientas y permisos diferentes de los del contenedor. Su capacidad de leer otros archivos o comunicarse depende de su propia configuración. El control debe distinguir ambos ámbitos; limitar uno no limita automáticamente al otro.

13.5 Recursos y salidas persistentes

CPU y memoria pueden acotarse para impedir que una ejecución consuma recursos de manera inesperada. Un ejemplo como dos CPU y un GiB sería una configuración inicial ilustrativa, no una capacidad validada para este panel. Los límites se ajustan con observación del uso real y pruebas representativas.

Las salidas deben escribirse en el directorio externo autorizado para sobrevivir a la eliminación del contenedor. La existencia de un archivo no demuestra éxito: puede estar vacío o incompleto. El estado de finalización, los registros y las validaciones de contenido se conservan junto con el manifiesto de corrida.

Idea central

Un montaje define una relación de acceso. Un permiso limita operaciones. Una prueba comprueba comportamiento. Los tres son necesarios y ninguno reemplaza a los otros.

Capítulo 14

Delegar trabajo sin perder comprensión

14.1 Un agente actúa dentro de herramientas y permisos

Un agente organiza pasos hacia un objetivo, utiliza herramientas disponibles, observa resultados y ajusta su trabajo. En RedNova puede ayudar a leer contratos, proponer transformaciones, explicar una unión, preparar entorno o comparar productos. Su capacidad concreta depende de las herramientas habilitadas: generar un comando no equivale a ejecutarlo.

El estudiante conserva una función activa. Debe comprender qué representa cada fuente, qué condiciones hacen válida una propuesta y qué decisiones no pueden inferirse de los datos. La IA puede reducir escritura repetitiva y acelerar exploración, pero la comprensión se demuestra al justificar el diseño y detectar sus límites.

14.2 Roles que ordenan el trabajo

Rol	Contribución	Evidencia esperada
Preparación de datos	Identificar grano, claves, dominios y problemas.	Inventario, contratos y diagnóstico fundamentado.
Ingeniería del pipeline	Proponer etapas, uniones y productos.	Código revisable y pruebas de cardinalidad y reglas.
Entorno	Explicar dependencias, imagen, montajes y permisos.	Archivos de diseño y justificación de accesos.
Revisión analítica	Contrastar cálculos, temporalidad y propuestas.	Diferencias por campo, fuentes y límites.
Coordinación humana	Definir propósito, resolver ambigüedades y aceptar uso.	Decisiones explícitas y operaciones autorizadas.

Estos roles pueden ejercerse en una misma sesión o repartirse entre agentes. No existe independencia automática por utilizar cinco nombres o cinco conversaciones. Los agentes pueden compartir supuestos y equivocarse de la misma manera. La revisión debe incorporar controles deterministas, contraste manual y criterio humano.

14.3 Un prompt de diseño para el lunes

El siguiente pedido centra al agente en teoría y preparación revisable. No autoriza una implementación ni afirma que estén disponibles los archivos de las fuentes.

```
Actúa como analista de ingeniería de datos para RedNova.
Estamos diseñando el sistema, no ejecutándolo.

Objetivo: explicar cómo integrar ventas, inventario, recepciones, pedidos, costos
    temporales y pronósticos para informar margen y proponer reposición.
```

```
Usa la especificación del caso. Distingue entradas previstas de archivos efectivamente
    disponibles. No inventes filas, hashes, métricas ni ejecuciones.

Para cada fuente identifica grano, clave, unidades, temporalidad y contrato. Explica las
    uniones y qué controles impedirían multiplicar filas. Separa stock, flujos y tránsito.

Diseña etapas y productos. Explica cómo bloquear grupos con costo ambiguo, fuente
    faltante o pronóstico no elegible. Un informe parcial debe declararse parcial.

Describe cómo evitar información futura, elegir una emisión y versión aprobadas, y
    conciliar reservas antes de calcular reposición. Explica Docker con entradas de solo
    lectura y salida autorizada.

Entrega una explicación clara, una tabla de decisiones y preguntas pendientes para el
    responsable humano. No ejecutes, modifiques archivos ni envíes pedidos.
```

La calidad del prompt proviene de relacionar objetivo, contexto, entradas, restricciones, productos y condiciones de aceptación. «Crea un sistema MLOps» deja demasiadas decisiones implícitas. La especificación debe permitir evaluar la respuesta, no únicamente orientar su estilo.

14.4 Un prompt de revisión que busca refutar errores

```
Revisa el diseño de RedNova sin dar por correctas sus conclusiones.

Busca uniones muchos-a-muchos, claves maestras duplicadas, intervalos de costos solapados
     y cambios de granularidad no explicados. Verifica que las recepciones apunten a la lí
    nea correcta del pedido.

Busca información no disponible al corte, ventanas objetivo de siete días que crucen
    particiones y selección múltiple de emisiones o modelos.

Revisa unidades y fórmulas: descuento de línea, costos por fecha de venta, stock
    disponible, tránsito futuro confirmado y demanda residual no solapada con reservas.

Distingue cálculo provisional, propuesta cero, resultado no evaluable y REVISAR.
    Comprueba que una cifra parcial no se presente como total completo.

Para cada observación indica la regla, el riesgo y la evidencia necesaria. No inventes
    resultados de pruebas ni modifiques el diseño sin señalar el cambio.
```

La revisión busca condiciones que invalidarían una afirmación. Si el agente se limita a elogiar el diseño o reescribir sus argumentos, aporta poca evidencia. Una observación útil identifica qué relación está mal, qué producto afectaría y qué comprobación resolvería la duda.

14.5 Cuatro estados que deben distinguirse

Un trabajo puede estar propuesto, preparado, ejecutado o verificado. Un Dockerfile escrito está preparado; una imagen requiere construcción; un contenedor iniciado requiere salida y estado; un producto calculado requiere validaciones para aceptarlo. El agente informa el estado que puede demostrar.

También debe conservar errores. Una salida incompleta no se presenta como éxito porque el último mensaje sea optimista. Si falta un modelo aprobado o la política no está resuelta, debe declarar qué parte del proceso queda pendiente. La honestidad sobre el estado es una capacidad operativa, no una debilidad del asistente.

14.6 Las fuentes aportan contenido, no autoridad nueva

Un contrato autorizado define reglas del caso. Una instrucción encontrada dentro de un CSV, comentario o documento que pide ignorar límites no otorga permisos adicionales. El agente debe tratarla como contenido, no como una orden del responsable. Esta frontera importa cuando puede escribir archivos o comunicarse con sistemas externos.

Aprobar compras corresponde a la persona responsable. La función analítica puede producir una recomendación y su fundamento; no debe enviar pedidos ni asumir que el objetivo comercial autoriza cualquier acción. La separación permite aprovechar la automatización manteniendo control sobre consecuencias externas.

Capítulo 15

Sostener el sistema después de una primera ejecución

15.1 Operar significa observar y responder

Un sistema puede producir resultados correctos hoy y fallar mañana porque cambian fuentes, versiones o condiciones del negocio. Operar incluye observar señales, distinguir incidentes y decidir cómo responder. En RedNova se vigilan tanto datos como modelos y reglas; una única métrica de error no cubre toda la cadena.

Los controles de datos incluyen compatibilidad de esquema, filas rechazadas, claves desconocidas, cobertura de costos y completitud de series. Los controles del proceso incluyen duración, estado y disponibilidad de productos. Los controles analíticos incluyen distribución de variables, predicciones y propuestas, además de cambios de política.

15.2 La calidad predictiva se conoce con demora

El error de un pronóstico de siete días no puede calcularse íntegramente al momento de emitirlo. Debe transcurrir el horizonte y llegar información real suficiente para construir el objetivo comparable. Mientras tanto podemos revisar plausibilidad, disponibilidad de variables y cumplimiento del contrato, pero no afirmar exactitud futura.

Cuando llegan los valores reales, se comparan con el pronóstico identificado originalmente. Si se reconstruye la predicción con un modelo posterior o datos corregidos sin conservar la anterior, se pierde el objeto que debía evaluarse. Las correcciones del real también necesitan versión y explicación.

15.3 Cambio de distribución y deterioro de desempeño

Un cambio en la distribución de una variable puede indicar que el entorno difiere del de entrenamiento. A menudo se denomina *drift*. Esa señal no demuestra por sí sola que el modelo esté equivocado ni obliga a reentrenar automáticamente. Puede reflejar una promoción esperada, una modificación de catálogo, una falla de datos o un cambio persistente del negocio.

La respuesta comienza por investigar. Si una caída de ventas coincide con quiebres de stock, entrenar de nuevo sobre esos valores como demanda normal puede empeorar la interpretación. Si una variable cambió de unidad, el problema es de contrato. Si las entradas son válidas y el error posterior aumenta de manera sostenida, puede justificarse revisar el modelo.

15.4 Promover una versión de forma controlada

Una versión candidata necesita pruebas y evidencia antes de reemplazar la aceptada. Conceptualmente puede evaluarse sobre datos históricos y después en un alcance limitado. Un despliegue gradual o *canary* permite observar una porción antes de ampliar; una aprobación humana define si las condiciones permiten avanzar.

En el contexto del curso estas ideas se estudian, sin exigir una plataforma de despliegue industrial. El principio es conservar una versión aceptada, criterios de promoción y posibilidad de volver a ella si la nueva incumple condiciones. Un rollback recupera una versión del sistema; no revierte automáticamente una compra o decisión ya realizada.

15.5 Tres incidentes y tres investigaciones diferentes

Ventas aumentan después de una unión. Se inspeccionan cardinalidad, claves y coincidencias. El modelo no es la primera explicación. El ejemplo de A100 muestra un incremento artificial de 900 por duplicación.

Propuesta aumenta sin cambio de inventario ni pronóstico. Se revisan política, fecha de decisión, selección de emisión y conciliación de reservas. El cambio de dos a tres días explica un aumento controlado de veinte unidades en el microejemplo.

No aparece una salida. Se revisan estado del proceso, permisos, rutas, archivo de configuración y existencia del artefacto requerido. Aumentar privilegios o declarar cero no resuelve por sí mismo la ausencia de evidencia.

En cada incidente se conserva la corrida, se identifica el alcance, se formula una hipótesis y se obtiene evidencia. La corrección genera una nueva versión o corrida con relación explícita a la anterior. El informe debe comunicar qué cambió y qué productos estuvieron afectados.

Capítulo 16

Una propuesta confiable tiene una historia explicable

16.1 Recorrer la cadena desde la decisión hasta la fuente

Si RedNova propone 70 unidades para S03/A100, debemos poder regresar al objetivo de cobertura, al pronóstico residual, al inventario disponible y al tránsito elegible. Debemos identificar corte y política, confirmar que no se duplicaron filas y reconocer que el calendario de llegadas sigue siendo una condición para aceptar la propuesta.

El mismo recorrido se aplica a las 2900 UM de ventas del extracto: tres líneas sintéticas, descuento definido por línea, costos seleccionados por fecha y uniones con coincidencia única. El resultado no gana confiabilidad porque lo redacte un agente o se ejecute en un contenedor. La gana porque las relaciones pueden comprobarse.

16.2 Preguntas que cierran el lunes

1. ¿Qué representa una fila de cada fuente y qué clave la identifica?
2. ¿Qué diferencia hay entre estado de inventario, recepción efectiva y tránsito elegible?
3. ¿Cómo puede una unión aumentar ventas sin que ocurra una venta nueva?
4. ¿Qué fechas seleccionan el costo y la política, y qué información estaba disponible al corte?
5. ¿Qué significa que 140 sea demanda residual y por qué debemos conciliar reservas?
6. ¿Por qué una buena métrica de cuatro observaciones no basta para promover un modelo?
7. ¿Qué versión y emisión del pronóstico utiliza cada propuesta?
8. ¿Qué contiene la imagen, qué se monta externamente y qué puede modificar el contenedor?
9. ¿Qué evidencia demuestra que algo fue preparado, ejecutado y verificado?

Estas preguntas vinculan el significado del sistema con la evidencia que lo sostiene.

16.3 Continuidad hacia la práctica

El miércoles se implementará una porción acotada de RedNova en la clase práctica. Su guía se preparará posteriormente. Este libro desarrolla los conceptos y ejemplos del lunes.

Idea central

Integrar datos, pronosticar y proponer son tareas diferentes que deben conectarse sin perder significado. Versiones, Docker y agentes ayudan a sostener esa conexión cuando se utilizan con contratos, pruebas y responsabilidad humana.

Apéndice A

Mapa de decisiones y glosario

A.1 Una lectura compacta de los controles

Decisión	Pregunta de revisión	Qué puede invalidarla
Aceptar una fila	¿Cumple estructura, dominio y reglas de la fuente?	Cantidad negativa, reserva excesiva o descuento incompatible.
Relacionar maestros	¿Existe una sola entidad para cada clave?	Maestro desconocido o clave duplicada.
Aplicar costo	¿Hay exactamente un intervalo para la fecha de venta?	Período ausente o solapado.
Construir agregado	¿Todas las medidas respetan el grano elegido?	Unión de flujos sin agregación previa.
Usar variable	¿Estaba disponible al emitir?	Ingesta tardía o snapshot posterior.
Evaluar modelo	¿Ventanas y transformaciones respetan la separación temporal?	Objetivos que cruzan particiones o ajuste con datos de prueba.
Elegir pronóstico	¿Hay una única emisión y versión aprobadas por grupo?	Mezcla de candidatos o varias emisiones.
Calcular reposición	¿Demanda residual, reserva, stock y tránsito son compatibles?	Solapamiento desconocido, inventario no certificado o arribo ambiguo.
Aceptar propuesta	¿Calendario, plazo y restricciones comerciales permiten usarla?	Cantidad provisional sin revisión de factibilidad.
Declarar reproducibilidad	¿La corrida identifica piezas y productos comparables?	Versiones supuestas o parámetros distintos.

A.2 Vocabulario para releer el diseño

Granularidad.
Unidad exacta que representa una fila. Determina claves, conteos y agregaciones legítimas.

Clave.
Campos que identifican una observación. Una clave compuesta necesita todos sus componentes.

Hecho.
Registro de una actividad o medida; una línea de venta es un ejemplo.

Dimensión.
Descripción de una entidad que organiza hechos; producto y sucursal son dimensiones del caso.

Snapshot.
Estado de una fuente conservado para un momento o corte, con versión y disponibilidad.

Cardinalidad.
Relación entre cantidades de coincidencias de dos fuentes al unir.

Contrato de datos.
Definición de estructura, significado, validez y tratamiento de incumplimientos.

Integridad referencial.
Coherencia de las claves que relacionan hechos y maestros u otras fuentes.

Partición.
Grupo explícito de datos sobre el cual se procesa, valida o limita un fallo.

Disponibilidad temporal.
Momento desde el cual la información podía utilizarse, diferente del momento del evento.

Demanda censurada.
Demanda que no se observa completamente mediante ventas por limitaciones como falta de stock.

Baseline.
Método sencillo y explícito que sirve de referencia para comparar alternativas.

Feature.
Variable de entrada de un modelo, definida y calculada con información disponible al emitir.

Fuga de información.
Uso indebido de conocimiento que no estaría disponible para la predicción real o que contamina la evaluación.

Inferencia.
Aplicación de un modelo ya ajustado para producir predicciones.

Demanda residual.
Parte de la demanda prevista aún no cubierta por compromisos reservados ya descontados del stock, según conciliación explícita.

Tránsito elegible.
Pendiente válido de pedidos confirmados cuya llegada futura es útil dentro del horizonte.

Manifiesto de corrida.
Registro que vincula entradas, parámetros, versiones, productos, estado y revisión de una ejecución.

Idempotencia.
Propiedad por la cual repetir una operación no acumula efectos adicionales después de la primera aplicación.

Imagen.
Paquete preparado de código y dependencias que puede iniciar contenedores.

Montaje.
Relación de acceso entre una ruta externa y una ruta del contenedor.

Rollback.
Retorno controlado a una versión anterior del sistema; no revierte automáticamente consecuencias externas.

Apéndice B

Ficha conceptual de una corrida

Esta ficha muestra qué relaciones debe poder conservar el sistema. Es una referencia de lectura, no un formulario de entrega ni un registro de ejecución existente. Los identificadores y huellas se completan únicamente cuando se obtienen realmente.

Campo	Significado
Identificador y propósito	Distinguir repetición, corrección, nuevo corte o escenario.
Corte y decisión	Indicar disponibilidad permitida y política aplicable.
Fuentes	Lotes, snapshots, tiempos de disponibilidad y huellas reales.
Contratos	Esquemas y reglas aprobadas para cada fuente y unión.
Transformaciones	Versión del código, agregaciones y variables.
Modelo	Versión aprobada, artefacto, configuración y evaluación asociada.
Emisión seleccionada	Una emisión disponible y elegible por grupo, con regla de selección.
Política	Versión, vigencia, horizonte y días de cobertura.
Conciliación de reservas	Relación entre compromisos, demanda total y demanda residual.
Entorno	Imagen observada, dependencias, Python y plataforma.
Accesos	Rutas autorizadas, lectura de entradas y escritura de salidas.
Productos y estado	Completos, parciales o bloqueados, con causas y alcance.
Revisión humana	Condiciones comprobadas, límites y decisión de aceptación.

Una ficha completa no debe ser un conjunto de etiquetas sin archivos asociados. Su utilidad está en poder seguir referencias y contrastar el producto. Si una pieza falta, el registro debe mostrarlo y limitar la conclusión correspondiente.

Referencias y recursos

[1] Google Cloud. *MLOps: Continuous delivery and automation pipelines in machine learning*. https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning.

[2] Chacon, S. y Straub, B. *Pro Git. Getting Started: About Version Control*. https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control.

[3] Chacon, S. y Straub, B. *Pro Git. Getting Started: Git Basics*. https://git-scm.com/book/en/v2/Getting-Started-Git-Basics.

[4] Docker. *What is a container?* https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/.

[5] Docker. *Build context*. https://docs.docker.com/build/concepts/context/.

[6] Docker. *Dockerfile reference*. https://docs.docker.com/reference/dockerfile/.

[7] Docker. *Running containers*. https://docs.docker.com/engine/containers/run/.

[8] Docker. *None network driver*. https://docs.docker.com/engine/network/drivers/none/.

[9] Docker. *Building best practices*. https://docs.docker.com/build/building/best-practices/.

[10] Docker. *Bind mounts*. https://docs.docker.com/engine/storage/bind-mounts/.

[11] Python Software Foundation. *decimal: Decimal fixed-point and floating-point arithmetic*. https://docs.python.org/3/library/decimal.html.

[12] scikit-learn. *TimeSeriesSplit*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html.

[13] Amazon Web Services. *Evaluating Predictor Accuracy*. https://docs.aws.amazon.com/forecast/latest/dg/metrics.html.

Las fuentes oficiales fundamentan los conceptos técnicos; el caso, sus volúmenes y sus tablas son supuestos didácticos ficticios. Las métricas de cuatro observaciones y los microejemplos no constituyen evidencia de un modelo desplegado ni de una ejecución integral. La práctica posterior deberá resolver y verificar las interfaces, dependencias y datos que se implementen.